Git and GitHub Quick Reference

Pushing daily practice work to GitHub | Repo: qa-upskilling | Local folder: D:\qa-upskilling

1. The big picture

Git is a version-control tool that runs on your computer and records snapshots of your files. GitHub is a website that stores a copy of that history online. Getting your work from your D drive to GitHub always follows the same path:

Working folder files you edit	**Staging area** chosen for next snapshot	**Local repository** saved commits on your PC	**GitHub (remote)** online backup

`git add` moves files from the working folder to staging. `git commit` saves the staged files as a snapshot in the local repository. `git push` uploads those snapshots to GitHub.

2. One-time setup (do these once, in this order)

#	Command / Action	What it does	Why we need it
1	Install Git for Windows (from git-scm.com)	Installs the Git program on your computer.	Without it, the git commands do not exist. Default options are fine.
2	`git --version`	Prints the installed Git version.	Confirms the install worked before you go further.
3	`git config --global user.name "Your Name"` `git config --global user.email "you@example.com"`	Stores your name and email, which are stamped on every commit.	Git refuses to commit without an identity. Use the email linked to your GitHub account so commits are credited to you. Needed only once per computer.
4	`cd D:\qa-upskilling`	Moves the terminal into your project root folder.	All Git commands act on the folder you are in, so you must be in the root.
5	`git init`	Turns the folder into a Git repository by creating a hidden .git folder.	Tells Git to start tracking this folder. The .git folder is where Git keeps all history.
6	`git branch -M main`	Renames the current branch to main.	Matches the branch name GitHub uses, so local and remote line up.
7	`git remote add origin https://github.com/bagwansameer/qa-upskilling.git`	Links your local repo to the GitHub repo and names that link origin.	Git needs to know where to push. origin is the standard nickname for the main remote.
8	`git pull origin main` (only if the GitHub repo already has files, e.g. a README)	Downloads what is on GitHub and merges it into your local copy.	Makes local and GitHub histories match before you push. Skip it if the GitHub repo is completely empty.
9	Create a file named `.gitignore` in the root folder	Lists files and folders Git must never track.	See section 3. Do this BEFORE your first git add.

3. The .gitignore file (where, what and why)

Where: in the root folder, `D:\qa-upskilling`, next to the .git folder. A .gitignore only affects the folder it sits in and the folders below it, so placing it inside a subfolder such as TS Practice or Day_1 protects only that subfolder.

Content used in this project:

```
node_modules/
test-results/
playwright-report/
blob-report/
.env
```

#	Entry	What it is	Why it is ignored
1	node_modules/	Folder of installed npm packages.	Can contain thousands of files and is rebuilt any time with npm install. It should never be in the repo.
2	test-results/, playwright-report/, blob-report/	Output generated by Playwright test runs.	Generated files, not your work. They change on every run and only add noise.
3	.env	File that holds secrets such as passwords and API keys.	Secrets must never be pushed to a public repo.

How to create it: in VS Code, open the root folder (File > Open Folder > D:\qa-upskilling), click the New File icon in the Explorer, type exactly `.gitignore`, paste the content and save. Avoid Notepad: it often saves the file as .gitignore.txt, which Git ignores.

Common mistake in this project: the file was first created inside Day_1 and then inside TS Practice. The check is `git status`: .gitignore must appear as *.gitignore* with no folder path in front of it. To move it up one level from TS Practice: `move "TS Practice\.gitignore" .gitignore`.

4. The daily routine (after every practice session)

Always run these from the root folder, `D:\qa-upskilling`.

#	Command / Action	What it does	Why we need it
1	`git status`	Shows which files are new (red, untracked), staged (green) or changed.	A safety check before committing, so you know exactly what will be saved. Confirm node_modules does not appear.
2	`git add .`	Stages every new or changed file in the current folder and all subfolders.	Git only saves what you have staged. The dot means everything from here downward, so new folders are picked up automatically.
3	`git commit -m "Day 2: functions and interfaces"`	Saves the staged files as a snapshot with a message.	Creates a permanent, restorable checkpoint. Clear messages turn the repo into a readable learning log.
4	`git push`	Uploads new commits to GitHub.	Until you push, the work exists only on your PC. Pushing gives you a backup and a visible record of progress.

First push only:

```
git push -u origin main
```

The `-u` flag links your local main branch to origin/main, so from then on a plain `git push` works. On the first push, a browser window opens asking you to sign in to GitHub. Sign in once and Windows remembers it. GitHub no longer accepts your account password in the terminal, which is why browser sign-in is used.

Order matters: add, then commit, then push. You cannot commit files that are not staged, and you cannot push commits that do not exist.

5. Adding a new folder (for example, Playwright Practice)

#	Command / Action	What it does	Why we need it
1	Create the folder inside D:\qa-upskilling and put at least one file in it	Makes the folder something Git can see.	Git tracks files, not folders. An empty folder will not appear in git status or on GitHub.
2	`git status`	Lists the new folder's files in red as untracked.	Confirms Git can see them.
3	`git add .`	Stages the new folder along with any other changes.	No need to name the folder. To stage only one folder, use `git add "Playwright Practice"`.
4	`git commit -m "Add Playwright Practice folder"`	Saves the snapshot.	Records the addition with a descriptive message.
5	`git push`	Sends it to GitHub.	The folder now shows on GitHub next to TS Practice.

Playwright note: when you run `npm init playwright@latest` it creates node_modules, playwright-report and test-results. The root .gitignore already excludes them in every subfolder, so only real files (tests, playwright.config.ts, package.json, package-lock.json) are pushed.

6. Warnings and errors you may see

#	Message / symptom	What it means	Fix
1	warning: LF will be replaced by CRLF the next time Git touches it	Windows uses CRLF line endings, Git stores LF. Git converts automatically.	Harmless. Ignore it and continue.
2	Please tell me who you are	Git does not know your name and email yet.	Run the two git config commands from section 2, step 3, then repeat the commit.
3	rejected ... fetch first or unrelated histories	GitHub has a commit your local copy does not (often the README created with the repo).	Run `git pull origin main --allow-unrelated-histories`, save and close any merge message window, then push again.
4	New folder does not show in git status	The folder is empty.	Add a file to it. Git does not track empty folders.
5	node_modules appears in git status	.gitignore is misnamed or in the wrong folder.	Check the name is exactly .gitignore (no .txt) and that it sits in D:\qa-upskilling.

7. Handy extra commands

Command	Purpose
`git status`	Shows what is new, changed and staged. Run it whenever unsure.
`git log --oneline`	Shows your commit history, one line per commit.
`git restore --staged <file>`	Unstages a file you added by mistake (the file itself is untouched).
`git add "folder/name"`	Stages only one folder or file, useful for separate commits per topic.

8. Interview questions and short answers

Q: What is the difference between Git and GitHub?

Git is the version-control tool that runs locally and tracks changes. GitHub is an online service that hosts Git repositories so you can back up, share and collaborate.

Q: What does git init do?

It creates a hidden .git folder in the current directory, turning it into a Git repository so Git starts tracking it.

Q: What is the staging area, and why does it exist?

It is a holding area between your working folder and the commit. It lets you choose exactly which changes go into the next snapshot, so unrelated changes can be committed separately.

Q: What is the difference between git add, git commit and git push?

add stages changes. commit saves the staged changes as a snapshot in the local repository. push uploads local commits to the remote repository on GitHub.

Q: What is origin?

The default name for the remote repository you linked with git remote add. It is a nickname for the GitHub URL.

Q: What does the -u flag in git push -u origin main do?

It sets the upstream, linking local main to origin/main, so later you can run just git push or git pull without naming the remote and branch.

Q: What is .gitignore and why use it?

A file listing paths Git should not track, such as node_modules, generated reports and .env secrets. It keeps the repo small and clean and prevents leaking credentials.

Q: Why should node_modules not be committed?

It is huge, rebuilt by npm install from package.json and package-lock.json, and machine-specific. Committing it bloats the repo and slows everything down.

Q: Why are secrets like .env kept out of the repo?

Anyone with access to the repo could read them, and they stay in the history even if deleted later. They should be kept local or in a secrets manager.

Q: Why did an empty folder not show up on GitHub?

Git tracks file contents, not folders. A folder appears only when it contains at least one file.

Q: What is a good commit message?

Short and specific about what changed and why, for example: Day 5: TypeScript functions and interfaces. It makes history readable and easier to debug.

Q: What does git pull do, and when did you need it?

It downloads new commits from the remote and merges them into your local branch. It is needed when GitHub has commits your local copy lacks, such as a README created with the repo.

Q: How do you check what is about to be committed?

Run git status. Green files are staged for the commit, red files are changed or untracked but not staged.

Q: How do you unstage a file you added by mistake?

git restore --staged followed by the file name. The file stays as it is; it is only removed from staging.

9. One-page cheat sheet

One time only:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
cd D:\qa-upskilling
git init
git branch -M main
git remote add origin https://github.com/bagwansameer/qa-upskilling.git
(create .gitignore in the root folder)
```

Every practice session (from D:\qa-upskilling):

```
git status
git add .
git commit -m "Day N: what you practiced"
git push            (first time: git push -u origin main)
```